MLP-LAB · EDUCATIONAL

BERT/GPT Batch Processing & GPU Parallelism

Walk through Batch Size = 2 to understand how a GPU processes multiple samples at once

01

Batch Size = 2

Feed two samples in one pass

02

Parallel, not Concat

Side by side — not stitched together

03

Memory & Throughput

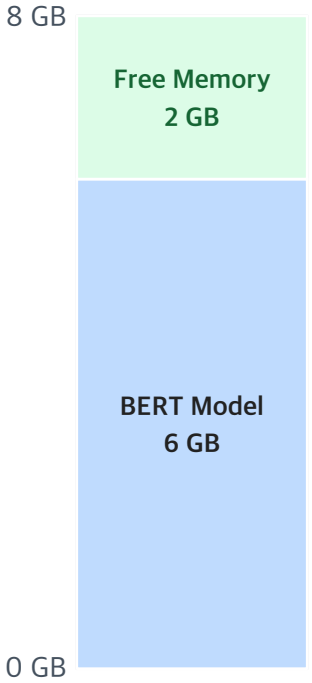
Balance speed against memory cost

MLP-lab.

GPU Memory Situation — An Assumption

With a 6 GB BERT or GPT model resident on an 8 GB GPU, how many samples fit at once?

GPU Memory (Total 8 GB)



Extra memory required per input sample

≈ 1 GB (forward / backward)

↓

✓ Maximum number of samples processed simultaneously

= 2 (Batch Size = 2)

💡 In practice, memory is also consumed by **parameters, activations, attention, gradients, and optimizer states**. The numbers shown here are simplified for illustration.

Batch Size ≠ Concatenating Tokens

Batching does not stitch two sequences into one — it runs both sequences side by side

Wrong · Concatenating Tokens

Sample A (512 tokens)

A1	A2	...	AS11	AS12

Sample B (512 tokens)

B1	B2	...	BS11	BS12

↓

1024 tokens (single sequence)

A1	A2	...	A_S12	B1	B2	...	B_S12

✗ The model treats it as one long sequence — attention crosses the boundary between samples

Right · Batch Processing (Parallel)

Sample A (512 tokens)

A1	A2	...	AS11	AS12

↓

Sample B (512 tokens)

B1	B2	...	BS11	BS12

✓ Two sequences stay independent and are processed simultaneously

MLP-lab.

Actual Input Tensor Structure

Batch Size = 2, Sequence Length = 512 — one row per sample, one column per token

Example: Batch Size = 2, Sequence Length = 512

Input Tensor Shape

[2, 512]

↕ Batch Dimension (number of samples)

↔ Sequence Length (number of tokens)

Token Position

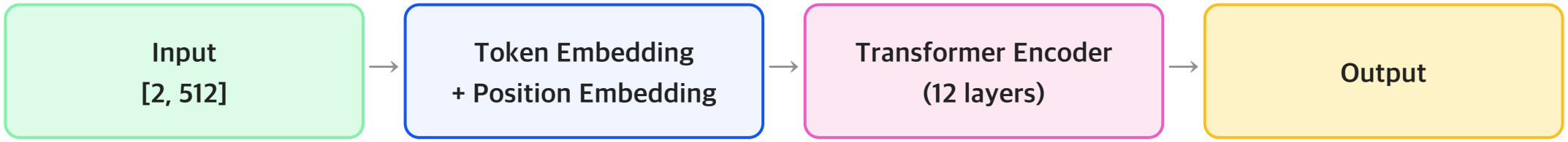
	1	2	3	...	510	511	512
Sample A	A1	A2	A3	...	AS10	AS11	AS12
Sample B	B1	B2	B3	...	BS10	BS11	BS12

⭐ Each sample occupies **exactly one row of the batch dimension** — the two samples never mix inside the same sequence.

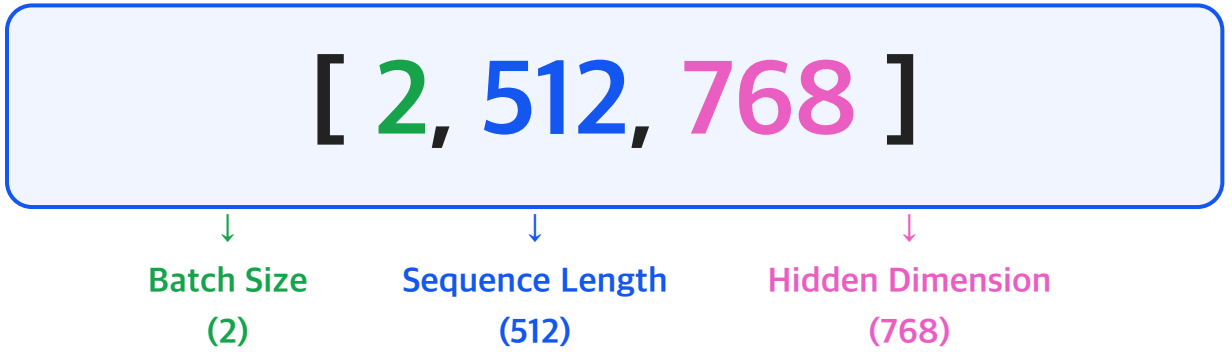
How BERT Works Internally

BERT runs on large matrix ops — both samples flow through every layer in parallel on the GPU

BERT is based on large-scale matrix operations



The hidden states have shape [2, 512, 768] (BERT-base, hidden dim = 768)

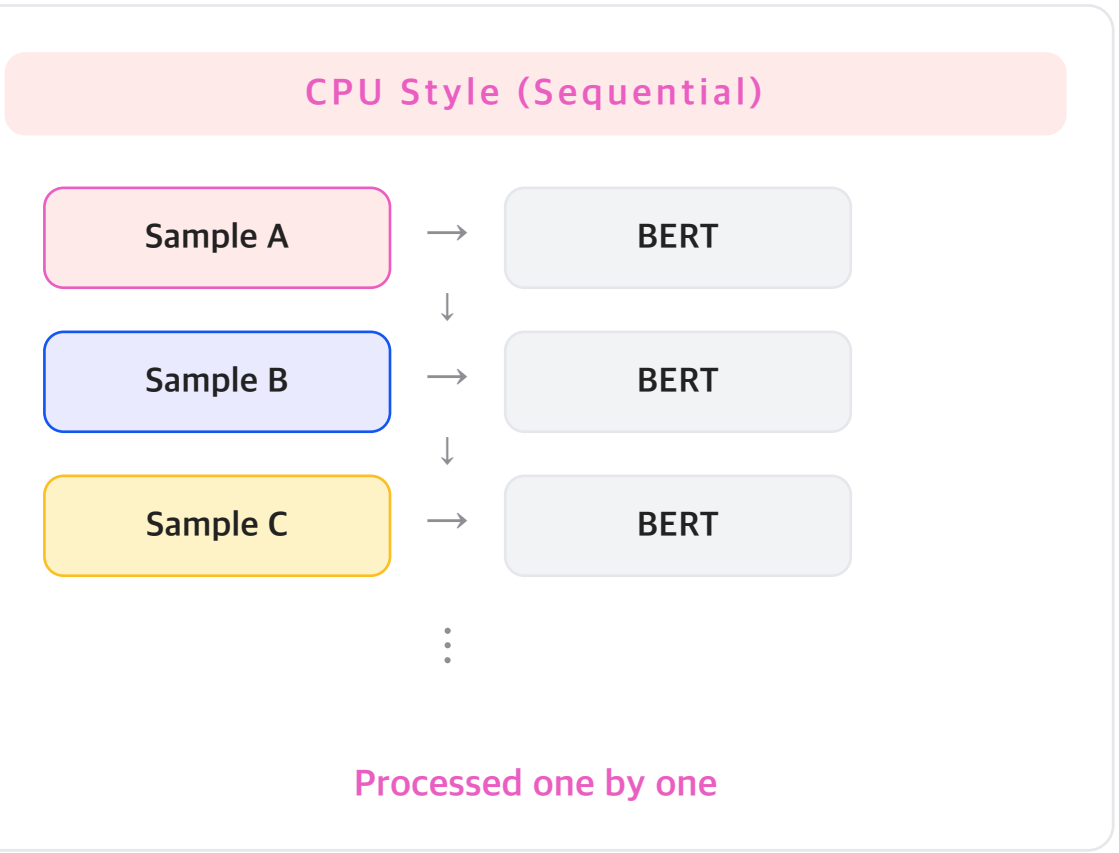


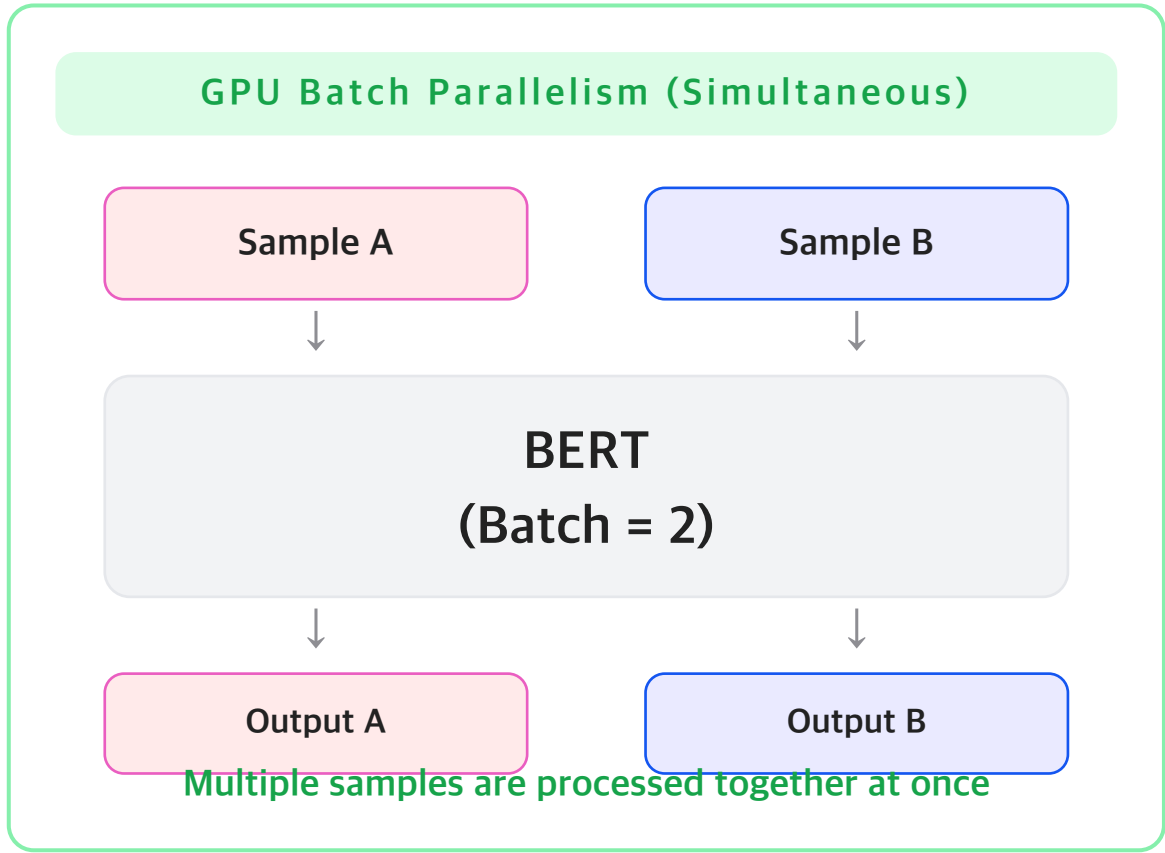
- Both samples are processed in parallel at every layer on the GPU
- Each sample is computed independently, but the ops run simultaneously

MLP-lab.

CPU Processing vs GPU Batch Parallelism

One at a time vs many at once — the same work is done in fundamentally different ways





Why Use a Larger Batch Size?

Throughput and efficiency rise — so does memory pressure. The trick is finding the right balance

Advantages

- ✓ Higher GPU utilization
- ✓ Higher throughput
- ✓ Faster training
- ✓ Better computational efficiency

Disadvantages

- ! More GPU memory usage
- ! Risk of OOM (out of memory)
- ! More activation storage

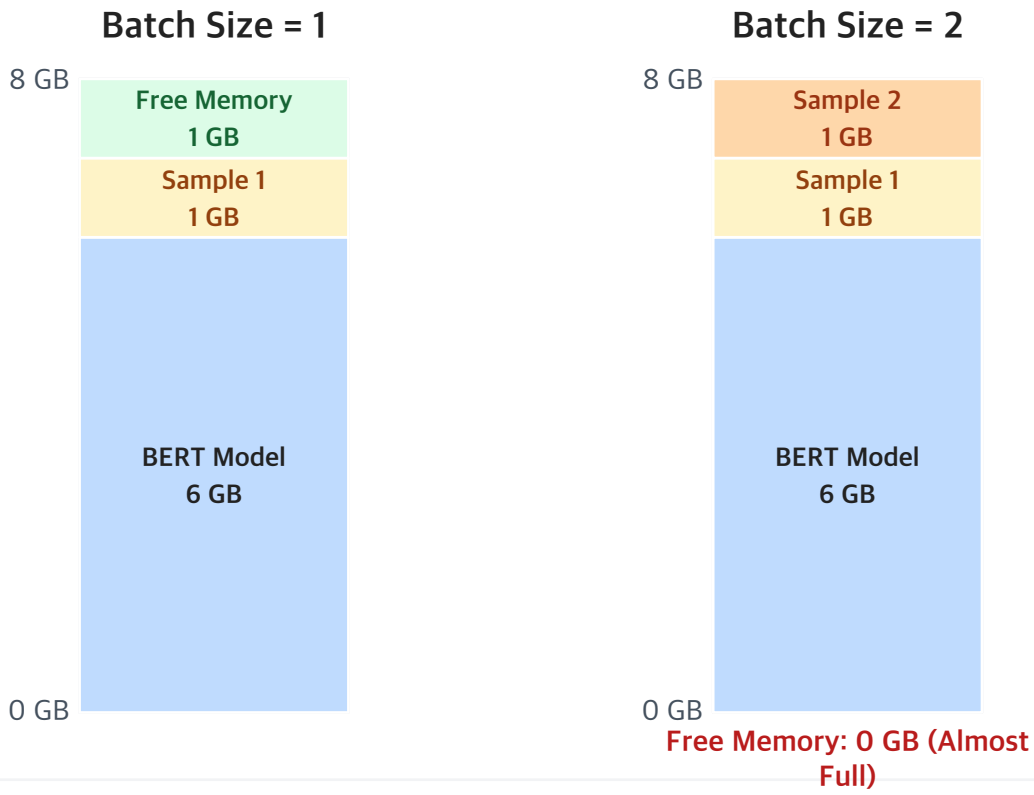
⭐ **Finding the right batch size is critical.**

Too small → underutilization · Too large → out of memory (OOM)

MLP-lab.

Memory Usage Example — Batch Size 1 vs 2

The 6 GB model is fixed — what changes is how many samples fit in the memory left over



⭐ Key Point

The model (6 GB) consumes the bulk of GPU memory. The memory that remains determines **how many samples can be loaded simultaneously.**

Increase the batch size, and **more memory is required.**

Summary

Batch Size ↔ GPU Memory ↔ Parallel Processing — understanding this relationship is the key

1. BERT processes multiple sentences in a batch — not by concatenating their tokens.
2. Batch Size = 2 means the input tensor shape is [2, 512] or [2, 512, hidden_dim].
3. The GPU processes multiple samples in parallel, not sequentially.
4. GPUs are designed for large matrix operations to maximize efficiency.
5. A larger batch size improves speed, but requires more GPU memory.

🎯 **Batch Size ↔ GPU Memory ↔ Parallel Processing**

Understanding this relationship is the key.

MLP-LAB · EDUCATIONAL

BERT/GPT
Full-Tuning vs LoRA vs QLoRA

Walk through Batch Size = 2 to understand how a GPU processes multiple samples at once

01

Batch Size = 2

Feed two samples in one pass

02

Parallel, not Concat

Side by side — not stitched together

03

Quantization

Balance speed against memory cost

MLP-lab.

BERT Fine-Tuning Memory: Full FT vs LoRA vs QLoRA

A simple GPU memory view with a 6 GB pretrained BERT assumption

8 GB GPU Memory (Total)

Pretrained weights alone consume 75% of an 8 GB GPU before training even begins.

Memory categories during training

1. **Model Params** — Stored weights
2. **Activations** — Hidden states
3. **Attention** — Score matrices
4. **Gradients** — For trainable params
5. **Optimizer** — Adam states (m, v)

1 Training needs more than weights

Activations, gradients, and optimizer states all consume memory.

2 LoRA freezes the base model

Only small adapter weights are trainable, slashing gradient + optimizer memory.

3 QLoRA quantizes to 4-bit

Reduces base-model storage on top of LoRA's savings.

Teaching example — illustrative memory figures, not benchmark values.

MLP-lab.

What Consumes GPU Memory During Training?

Teaching assumption: 6 GB pretrained BERT on an 8 GB GPU

1

Model Parameters

The stored weights of the pretrained model — the largest fixed cost.

2

Activations

Intermediate hidden states kept in memory for backpropagation.

3

Attention Matrices

Attention scores and probabilities; grows with sequence length squared.

4

Gradients

Required for every trainable parameter during backpropagation.

5

Optimizer States

Extra states such as Adam momentum (m) and variance (v) for each param.

Illustrative batch size = 2

Activations	≈ 1.4 GB
Attention matrices	≈ 0.6 GB
Total intermediate memory	**≈ 2.0 GB**

Important

- These are approximate teaching numbers.
- Real values depend on dtype, sequence length, optimizer, and implementation.
- Earlier simplified examples often hide these categories inside generic 'sample memory'.

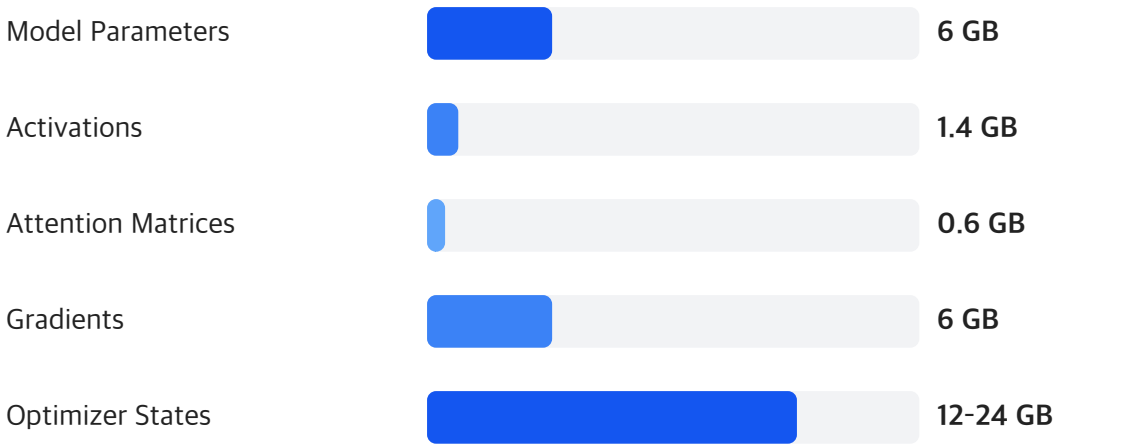
Teaching example — figures are illustrative.

MLP-lab.

Full Fine-Tuning: Why Memory Explodes

All model weights are trainable — every component scales with the full model

8 GB GPU Memory — Full FT Breakdown



Total ≈ 26-38 GB (4-5× over 8 GB budget)

1 Why so large?

Gradients are needed for **every trainable parameter**, and Adam/AdamW adds optimizer states (m, v) for the entire model — roughly 2× more memory.

2 What is trainable?

All blocks are trainable → full backward pass

3 8 GB GPU feasibility

⚠ **Not feasible under this assumption.** Memory required is 3-5× the GPU budget.

Numbers assume Adam/AdamW + fp16 mixed precision; illustrative.

MLP-lab.

LoRA Fine-Tuning: Freeze the Base Model

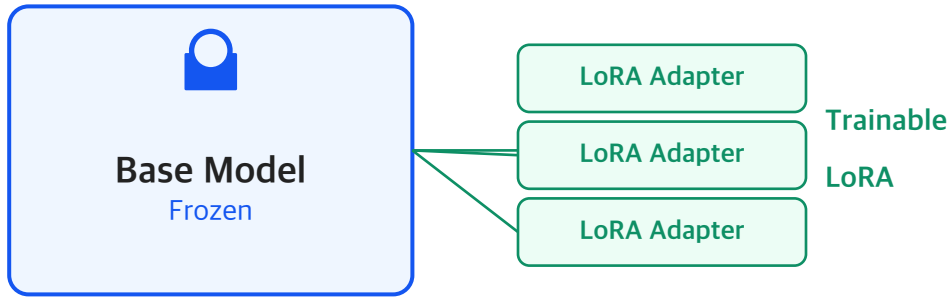
Only small adapter weights are trainable — gradients and optimizer states shrink dramatically

GPU Memory Breakdown — LoRA

Frozen Base Model	6 GB
LoRA Adapters	**0.3 GB (≈5%)**
Activations	1.2-1.4 GB
Attention Matrices	0.6 GB
Gradients	**0.3 GB**
Optimizer States	**0.6-1.2 GB**

Total ≈ 9.0-9.8 GB (~3× smaller than Full FT)

Model Structure (Conceptual)



↓ What gets reduced?

- Base-model gradients removed
- Optimizer states (m, v) removed
- Trainable params drop ~95%

✓ What stays?

- Base-model parameters still in memory
- Activations + attention still needed
- Forward pass identical to Full FT

LoRA: Low-Rank Adaptation (Hu et al., 2021) — figures are illustrative.

MLP-lab.

QLoRA Fine-Tuning: Add 4-Bit Quantization

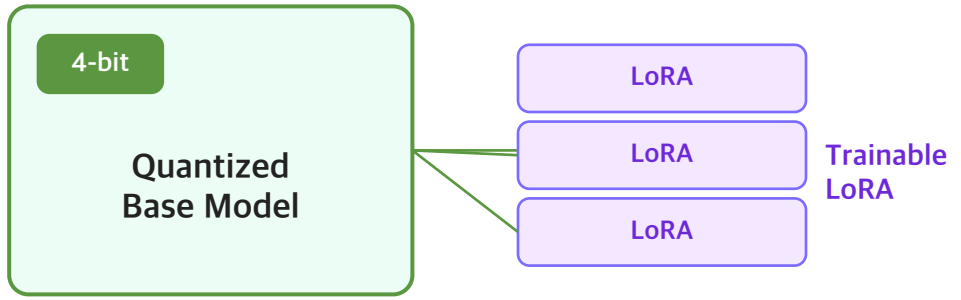
Quantized base model + trainable LoRA adapters — base-model storage shrinks too

GPU Memory Breakdown — QLoRA

4-bit Base Model	1.5-1.8 GB
LoRA Adapters	0.3 GB
Activations	1.2-1.4 GB
Attention Matrices	0.6 GB
Gradients	0.3 GB
Optimizer States	0.6-1.2 GB

Total ≈ 4.5-5.6 GB (fits in 8 GB GPU)

Conceptual View



↓ **What reduces?** Base-model parameters · base-model gradients · base-model optimizer states

⏱ **Speed note** Per-sample compute may be slightly slower than LoRA because quantization/dequantization adds overhead, but a smaller memory footprint allows larger batch sizes.

QLoRA: Efficient Finetuning of Quantized LLMs (Dettmers et al., 2023).

MLP-lab.

Comparison at a Glance

Memory components for Full FT, LoRA, and QLoRA — same model, three strategies

Memory Component	Full FT	LoRA	QLoRA
Model Parameters	6 GB	6.3 GB	1.8-2.1 GB
Activations	1.4 GB	1.2-1.4 GB	1.2-1.4 GB
Attention Matrices	0.6 GB	0.6 GB	0.6 GB
Gradients	6 GB	0.3 GB	0.3 GB
Optimizer States	12-24 GB	0.6-1.2 GB	0.6-1.2 GB
Total Memory	**26-38 GB**	**9.0-9.8 GB**	**4.5-5.6 GB**
What is Frozen?	Nothing	Base model	Quantized base
8 GB GPU Feasibility	Not feasible	Tight	Likely feasible

Key insight LoRA saves trainable-state memory (gradients + optimizer). QLoRA saves trainable-state memory and base-model storage.

Numbers are teaching estimates; actual values depend on model + config.

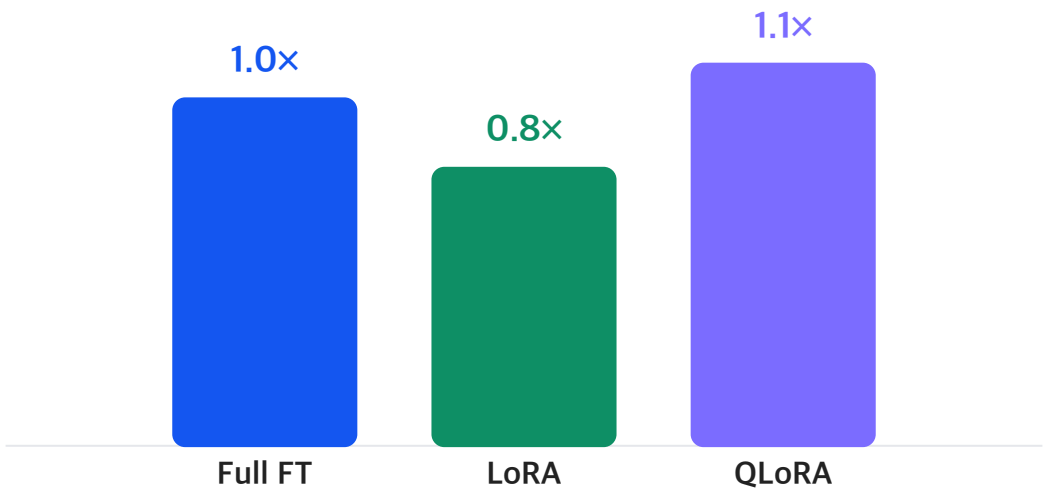
MLP-lab.

Training Speed: Step Time vs Total Throughput

Memory efficiency does not always mean faster per-sample compute

A. Same Batch Size = 2

Relative step time (lower is better)



B. Under an 8 GB GPU Limit

Larger batch sizes fit when memory is freed — total throughput can win.

Strategy	Batch	Steps / 100	Rel. time
Full FT	—	Not feasible	—
LoRA	≈ 1	100	≈ 80
QLoRA	≈ 4	25	≈ 30

Takeaway QLoRA may be **slower per step** than LoRA, but **larger batch sizes** can improve total throughput.

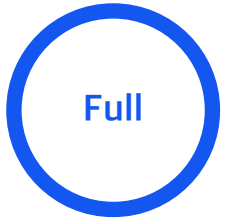
All speed numbers are conceptual teaching examples, not benchmark measurements.

MLP-lab.

Key Takeaways

What students should remember about Full FT, LoRA, and QLoRA

1



Full Fine-Tuning

All weights are trainable, so parameters, gradients, and optimizer states are all large. Often infeasible on small GPUs without aggressive optimization.

2

LoRA

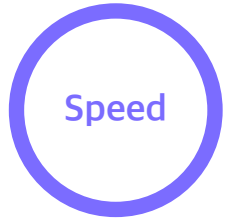
Freeze the base model and train small adapters. Greatly reduces gradient and optimizer-state memory while keeping the forward pass intact.

3

QLoRA

Quantize the base model to 4-bit and train LoRA adapters on top. Further reduces base-model storage, enabling fine-tuning on consumer GPUs.

4



Speed

LoRA is often faster per step than Full FT. QLoRA may be slower per sample but can win on throughput by fitting larger batch sizes in memory.

Teaching example — illustrative numbers for intuition.